

TDT4200 — Exercise 0

Q1: *What is the purpose of a Makefile?*

A Makefile's purpose is to automate the building process of C programs. It's methodology is defining building targets, their dependencies, and the recipe to make said target. The recipe is only executed if the target does not already exist or if the existing target file is older than any of its dependencies. E.g.:

```
# to make the `main` target binary we need the `main` object file (main.o)
# and the dependency's object file (dependency.o).
# Execute `gcc -o main <all-the-deps>` to make it.
main: main.o dependency.o
    gcc -o main $^

# To make any `*.o` file you need its `*.c` file.
# Execute `gcc -c <source-file> -o <object-file>`
%.o: %.c
    gcc -c $< -o $@
```

The programmer would then run `make main` to create the executable binary.

Q2: *What is a pointer in C?*

A pointer in C is an identifier that does not hold the data itself, but rather the address at which the data is stored. This is commonly used when allocating heap memory through functions like `malloc`. A pointer identifier is syntactically noted using `*`, e.g. `char *my_char` instead of `char my_char`

Q3: *What does the -O3 flag do when compiling C code?*

The `-O1`, `-O2`, `-O3` (and more) flags are all flags for setting the level of optimization the compiler should apply. They are more a collection of other optimization flags. `-O2` applies all optimization flags of `-O1` plus some more. `-O3` adds `-O2` flags plus even more. The goal of the optimization flags slightly differ, but they all aim towards a faster, more efficient binary. More aggressive optimization levels may produce a larger target binary and the compilation process may use more memory and take longer.

Q4: *How can you pass a value by reference to a function in C?*

C does not have pass-by-reference. Instead what we can do (to essentially emulate pass-by-reference) is to pass the value's address as the value:

```
void add(int *a, int b) {
    // "add `b` to the location pointed to by `a`"
    *a += b;
}

int main(void) {
    int num = 4;

    // `&num` returns the address of `num`, instead of the value of `num`.
    int *pointer_to_num = &num;

    // `num` == 4

    add(pointer_to_num, 3);

    // `num` == 7

    return 0;
}
```